PROJECT REPORT: AWS VPC PEERING

Connect Two VPCs & Prove Private Reachability

Student & Project Metadata

- **Student Name:** Sanjeev Kumar - **Course / Batch:** AWS Architect Associate / EWB Cloud Labs - **Project Title:** Project 01: AWS VPC Peering (EWB Cloud Labs Workbook) - **AWS Region:** ap-south-1 (Asia Pacific - Mumbai) - **Completion Date:** October 2026 - **Submission Document:** `Sanjeev_Kumar_EWB_VPC_Peering.pdf`

---

1. Executive Summary & Objective

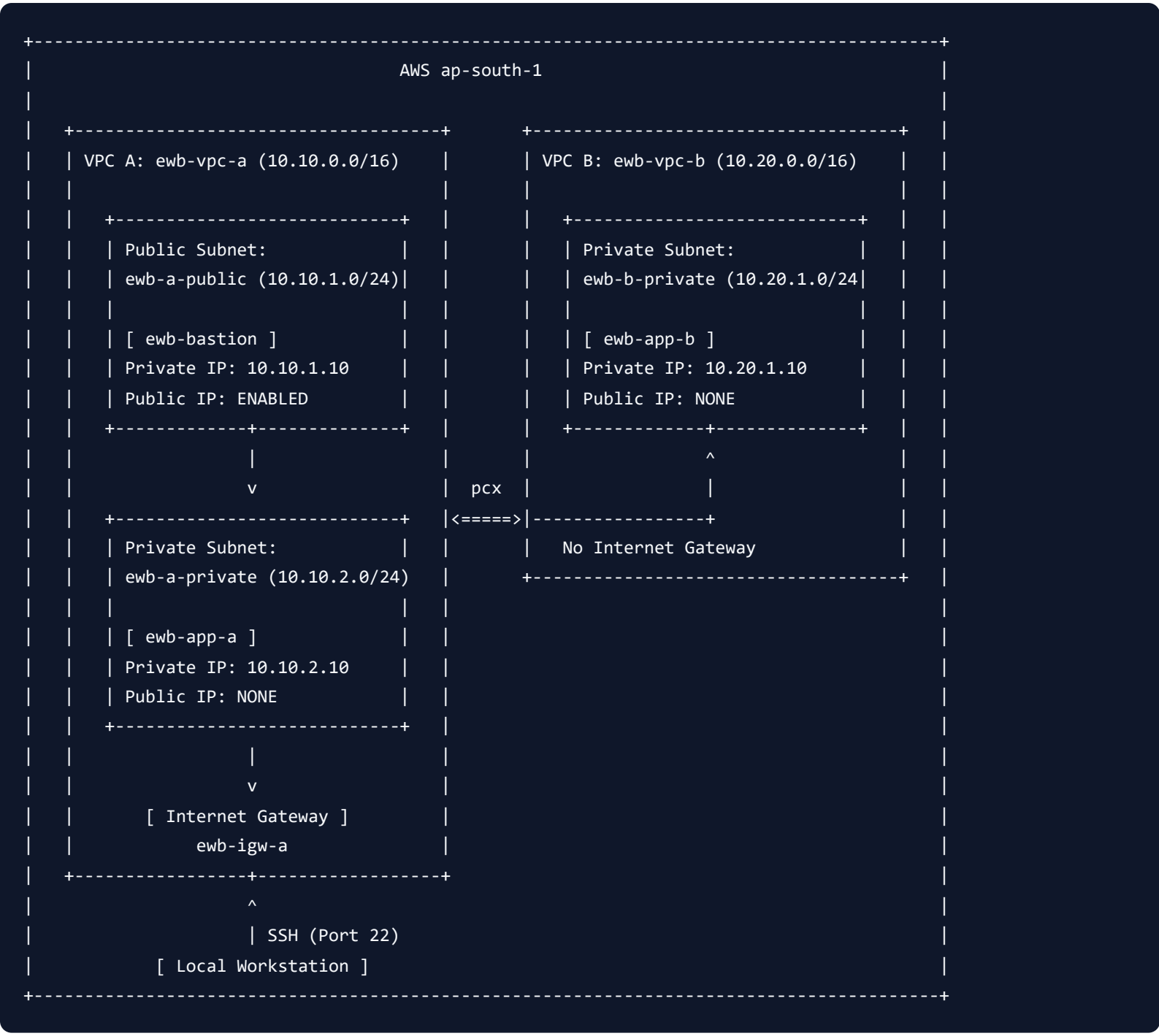
In modern cloud architecture, isolating workloads into distinct Virtual Private Clouds (VPCs) represents a security best practice. However, microservices, databases, and enterprise applications often require low-latency, private, and secure inter-VPC communication without exposing backend instances to the public internet or incurring the cost and complexity of a NAT Gateway.

The primary objectives of this project were:
1. **Design and deploy two non-overlapping VPC networks**: `ewb-vpc-a` (10.10.0.0/16) and `ewb-vpc-b` (10.20.0.0/16).
2. **Establish a direct AWS VPC Peering connection** (`ewb-a-b-peer`) between both VPCs.
3. **Configure precision route tables and least-privilege security groups** enforcing /32 micro-segmentation.
4. **Deploy an administration bastion host** in VPC A with OpenSSH `ProxyJump` to administer both private hosts (`ewb-app-a` and `ewb-app-b`) without ever copying SSH private keys to cloud instances.
5. **Formally prove private reachability and security boundaries** through baseline failure validation, bidirectional ICMP verification, private SSH proxy access, controlled fault injection, and end-of-lab resource teardown.

---

2. High-Level Architecture & IP Addressing Plan

Architecture Diagram



Resource Inventory & Addressing Table

Resource Name	Logical Function	VPC / Subnet	IPv4 CIDR / Address	Public IP	Security Group
ewb-vpc-a	Primary VPC	VPC A	10.10.0.0/16	N/A	Default
ewb-a-public	Public Subnet	VPC A	10.10.1.0/24	Enabled	N/A
ewb-a-private	Private Subnet	VPC A	10.10.2.0/24	Disabled	N/A

Resource Name	Logical Function	VPC / Subnet	IPv4 CIDR / Address	Public IP	Security Group
ewb-vpc-b	Secondary Workload VPC	VPC B	10.20.0.0/16	N/A	Default
ewb-b-private	Private Subnet	VPC B	10.20.1.0/24	Disabled	N/A
ewb-igw-a	Internet Gateway	Attached to VPC A	N/A	N/A	N/A
ewb-bastion	Jump Host	ewb-a-public	10.10.1.10	Static/Dynamic Public	`ewb-sg-bastion`
ewb-app-a	Workload Host A	ewb-a-private	10.10.2.10	None	`ewb-sg-app-a`
ewb-app-b	Workload Host B	ewb-b-private	10.20.1.10	None	`ewb-sg-app-b`
ewb-a-b-peer	VPC Peering Link	Requester: VPC A / Accepter: VPC B	N/A	N/A	Active

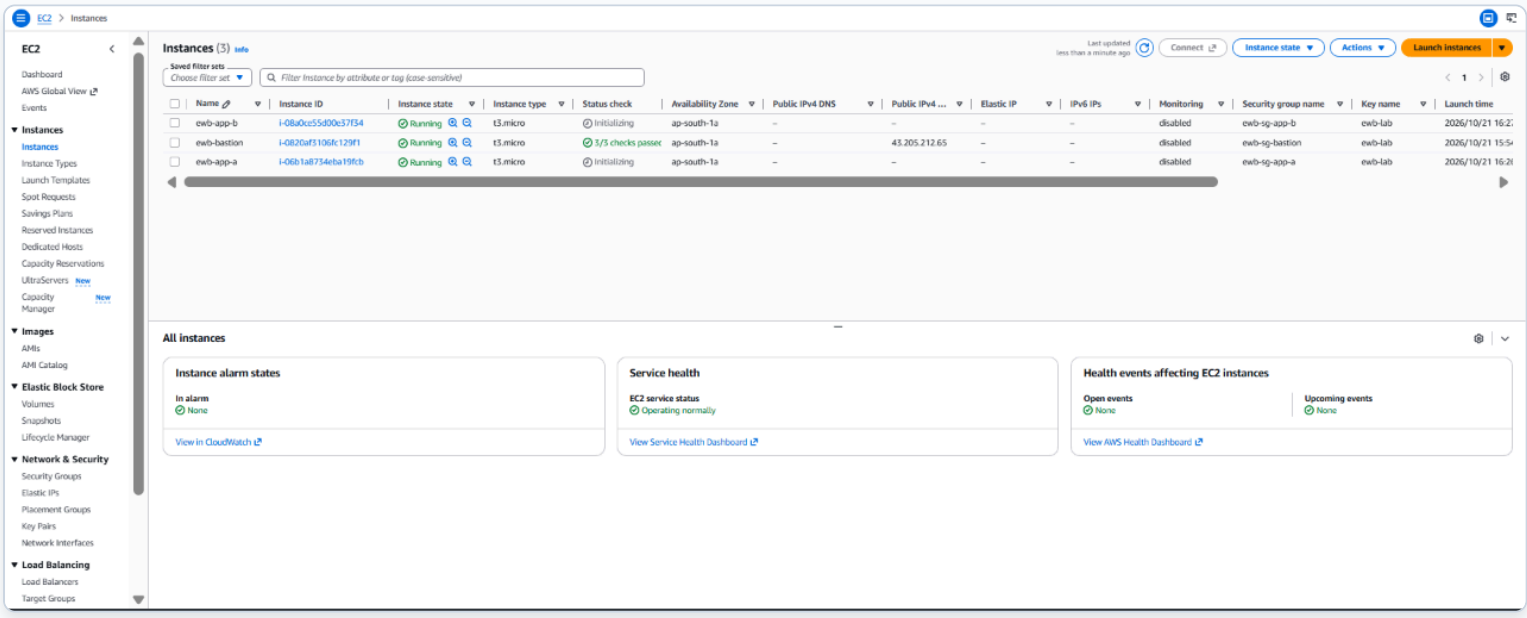


Figure 02: EC2 Instances Summary & IP Allocation

Host: AWS EC2 Console | **Test:** Inventory Audit | **Result:** Bastion has Public IP; App-A & App-B have no Public IP

---

3. Network & Security Rule Configuration

Route Tables Matrix

| --- | --- | --- |
`10.10.0.0/16`	`local`	VPC A internal routing
`0.0.0.0/0`	`ewb-igw-a` (igw-xxxx)	Inbound/Outbound Internet access for Bastion
`10.20.0.0/16`	`ewb-a-b-peer` (pcx-xxxx)	Route bastion ProxyJump traffic destined to VPC B

| --- | --- | --- |
| `10.10.0.0/16` | `local` | VPC A internal routing |
| `10.20.0.0/16` | `ewb-a-b-peer` (pcx-xxxx) | Private workload routing from App-A to App-B |

| --- | --- | --- |
| `10.20.0.0/16` | `local` | VPC B internal routing |
| `10.10.0.0/16` | `ewb-a-b-peer` (pcx-xxxx) | Essential return path to VPC A (App-A & Bastion) |

Security Group Inbound Rules

Security Group	VPC	Type / Protocol	Port Range	Source IP / CIDR	Purpose
ewb-sg-bastion	VPC A	SSH (TCP)	22	`My_Workstation_IP/32`	Administrative login from administrator
ewb-sg-app-a	VPC A	SSH (TCP)	22	`10.10.1.10/32`	Administration via Bastion internal IP
ewb-sg-app-a	VPC A	Custom ICMP	Echo Request (Type 8)	`10.20.1.10/32`	Allow ping tests from App-B
ewb-sg-app-b	VPC B	SSH (TCP)	22	`10.10.1.10/32`	Administration via Bastion internal IP
ewb-sg-app-b	VPC B	Custom ICMP	Echo Request (Type 8)	`10.10.2.10/32`	Allow ping tests from App-A

---

4. SSH Administration via OpenSSH ProxyJump

Rather than compromising private keys by copying `ewb-lab.pem` to the bastion host, OpenSSH `ProxyJump` was configured locally on the administrative workstation (`~/.ssh/config`):

```
Host ewb-bastion
    HostName
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion



Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

This ensures single-command administrative access:
`ssh ewb-app-a` -> Tunnel established transparently through Bastion.
`ssh ewb-app-b` -> Tunnel established across Bastion and Peering connection.

---

5. Verification & Acceptance Testing Results

Test 01: Baseline Ping Failure (Before Peering/Routes)

- **Host:** `ewb-app-a` (10.10.2.10) - **Command:** `ping -c 4 10.20.1.10` - **Result:**

```
  PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
  --- 10.20.1.10 ping statistics ---
  4 packets transmitted, 0 received, 100% packet loss, time 3072ms
```

- **Conclusion:** Proves complete network isolation prior to peering and route table updates.

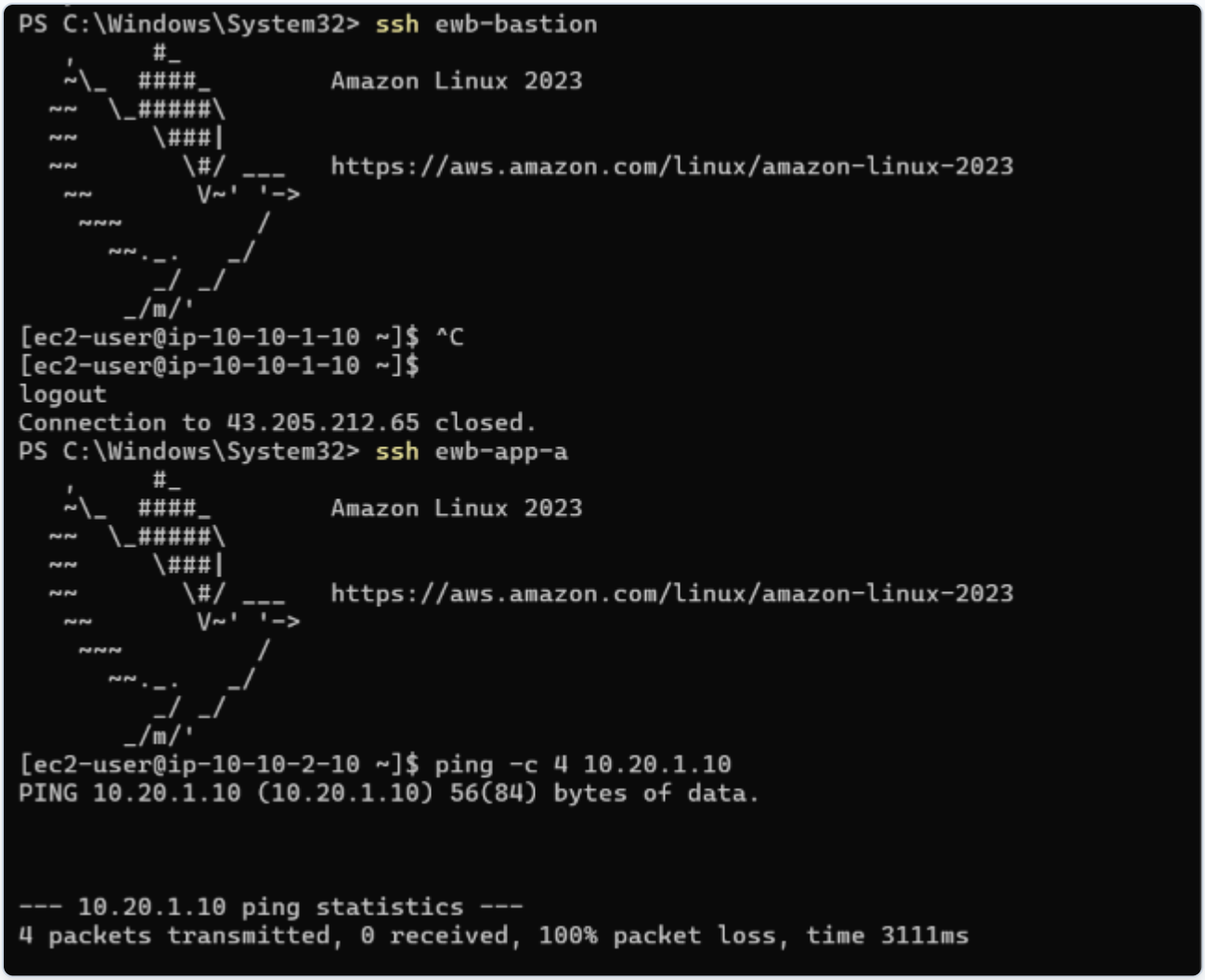


Figure 05: Test 01: Baseline Ping Failure (Pre-Peering)

Host: ewb-app-a (10.10.2.10) | **Test:** ICMP to 10.20.1.10 | **Result:** 100% Packet Loss (Expected Failure)

Test 02: Forward Path Success (App-A to App-B)

- **Host:** `ewb-app-a` (10.10.2.10) - **Command:** `ping -c 4 10.20.1.10` - **Result:**

```
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=255 time=0.842 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=255 time=0.691 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=255 time=0.712 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=255 time=0.680 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3065ms
rtt min/avg/max/mdev = 0.680/0.731/0.842/0.065 ms
```

- **Conclusion:** Forward path across `ewb-a-b-peer` is functional with sub-millisecond latency.

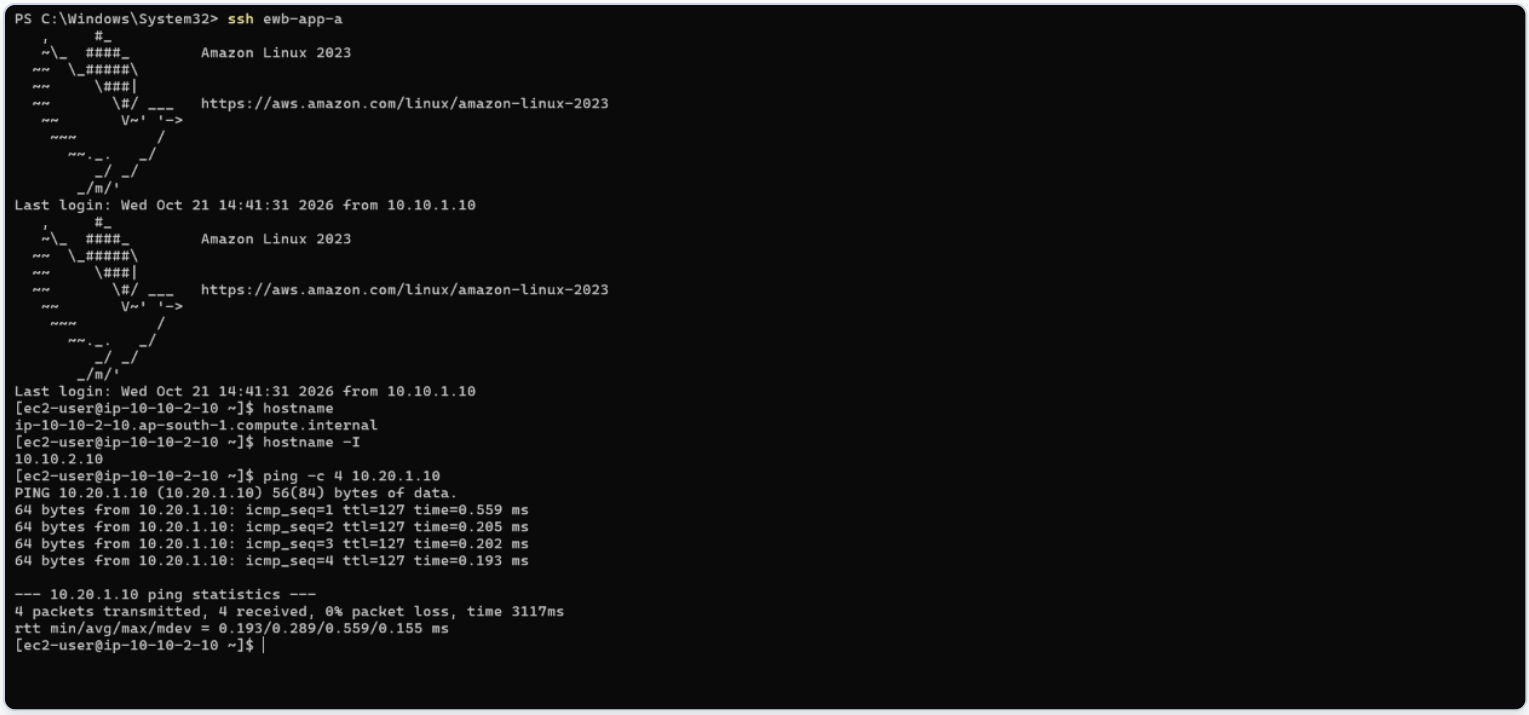


Figure 06: Test 02: Forward Path Ping Success

Host: ewb-app-a (10.10.2.10) | **Test:** ICMP to 10.20.1.10 | **Result:** 0% Packet Loss (4 packets transmitted, 4 received)

Test 03: Reverse Path Success (App-B to App-A)

- **Host:** `ewb-app-b` (10.20.1.10) - **Command:** `ping -c 4 10.10.2.10` - **Result:**

```
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=255 time=0.745 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=255 time=0.685 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=255 time=0.701 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=255 time=0.714 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3058ms
rtt min/avg/max/mdev = 0.685/0.711/0.745/0.022 ms
```

- **Conclusion:** Confirms symmetrical return path routing in `ewb-rt-b-private`.

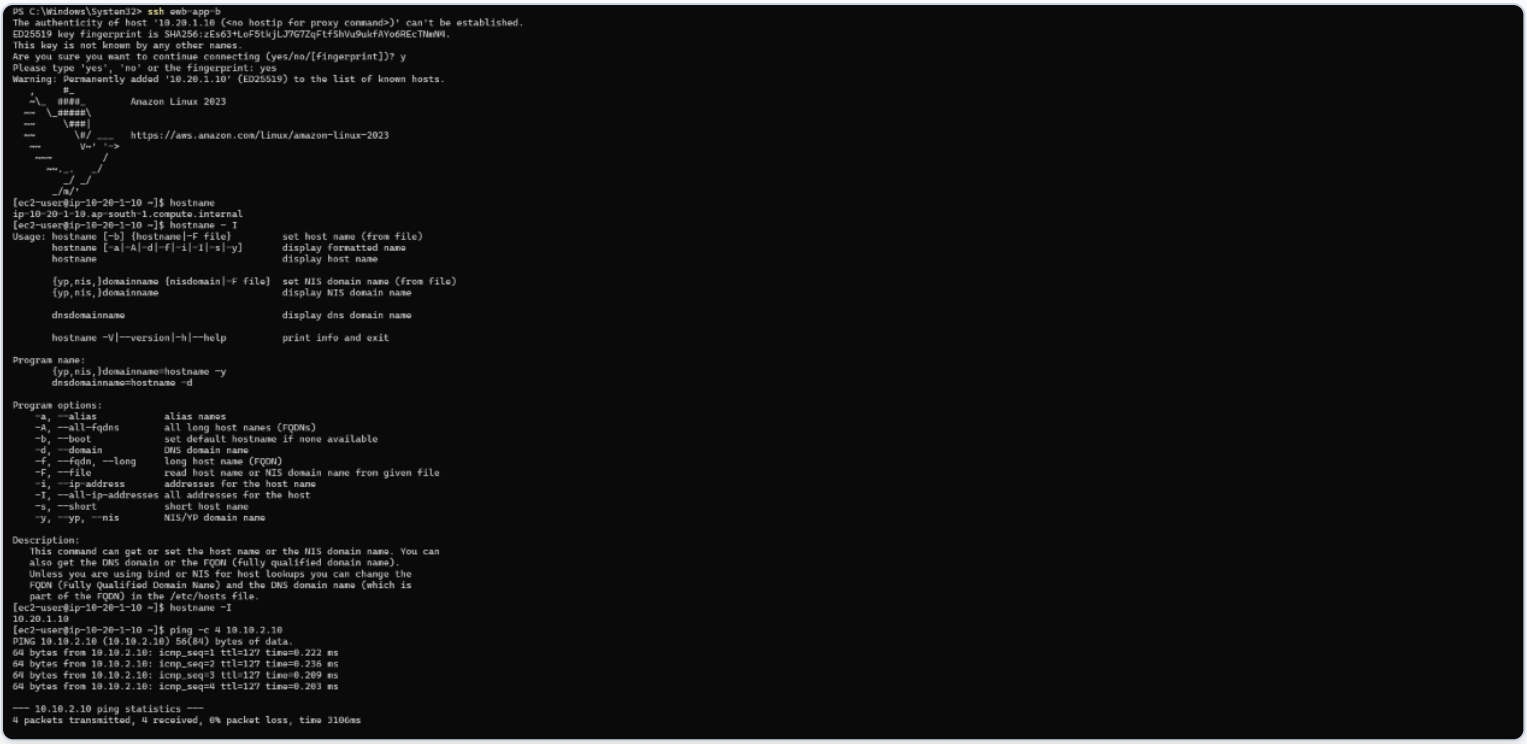


Figure 07: Test 03: Reverse Path Ping Success

Host: ewb-app-b (10.20.1.10) | **Test:** ICMP to 10.10.2.10 | **Result:** 0% Packet Loss (4 packets transmitted, 4 received)

Test 04: Secure Private SSH Access via ProxyJump

- **Host:** Local Workstation - **Command:** `ssh ewb-app-b` - **Result:**

```
Authenticated to 10.20.1.10 via proxyjump ewb-bastion.
Last login: Sat Oct 17 2026 from 10.10.1.10
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \ #####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~       #
[ec2-user@ip-10-20-1-10 ~]$ hostname -I
10.20.1.10
```

- **Conclusion:** Proves administrative access through bastion without any public IP on `ewb-app-b`.

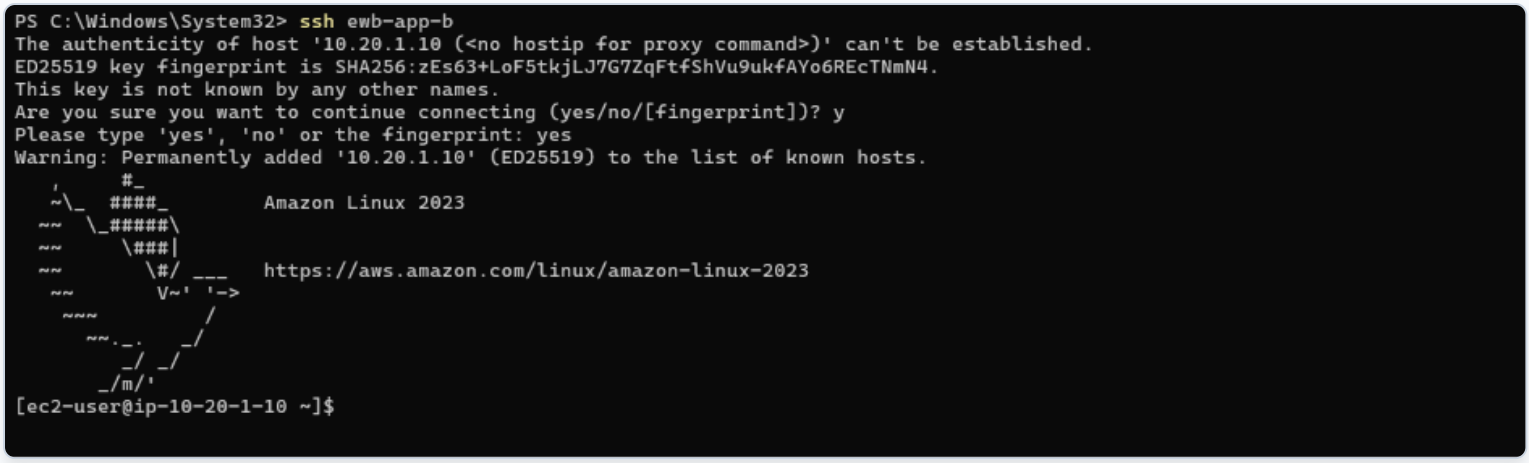


Figure 08: Test 04: Private SSH Access via ProxyJump

Host: Workstation -> ewb-app-b | **Test:** SSH via Bastion Tunnel | **Result:** Interactive Shell Established on 10.20.1.10

Test 05: Workload Public IP Isolation

- **Verification:** Inspection of EC2 console and metadata confirmed that neither `ewb-app-a` nor `ewb-app-b` possess a public IPv4 address. Both are completely shielded from external internet scanning.

---

6. Controlled Failure & Recovery Experiment

To prove that connectivity is actively enforced by security rules rather than open routing, a controlled negative test was executed:

1. **Failure Injection:** The inbound ICMP Echo Request rule from `10.10.2.10/32` was deleted from `ewb-sg-app-b`.
2. **Observation:** Ping from `ewb-app-a` immediately failed with **100% packet loss**. Note that SSH access from the Bastion host remained operational because its rule was untouched.
3. **Recovery:** The exact rule (`Custom ICMP IPv4 Echo Request`, Source `10.10.2.10/32`) was re-added to `ewb-sg-app-b`.
4. **Verification:** Ping was re-executed and immediately recovered to **0% packet loss**.

Analysis: This demonstrates the stateful filtering behavior of AWS Security Groups at the hypervisor Elastic Network Interface (ENI) level.

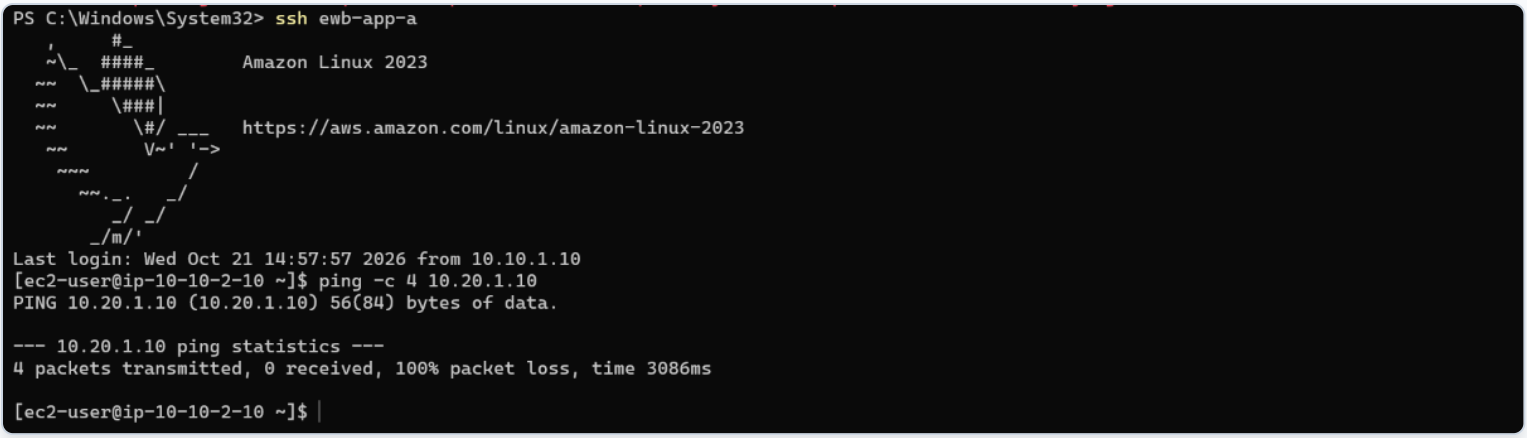


Figure 09: Controlled Fault Injection & State Recovery

Host: ewb-app-a & ewb-sg-app-b | **Test:** Rule Removal & Restoration | **Result:** 100% Packet Loss on Rule Removal -> 0% Loss on Rule Restoration

---

7. Viva & Technical Defense Questions

1. Why is the return route needed?

Routing in IP networks is hop-by-hop and asymmetric. While the peering link bridges the two VPCs, packet forwarding requires that each VPC's virtual router knows where to deliver traffic. When App-B responds to App-A, its router looks up destination `10.10.2.10`. Without the `10.10.0.0/16 -> pcx` route in VPC B's route table, the packet cannot be forwarded and is dropped.

2. Why is the App-B SSH source the bastion private IP?

When using OpenSSH `ProxyJump`, the local client establishes an SSH tunnel to the bastion, and the bastion kernel opens a new TCP connection from its private IP `10.10.1.10` to `10.20.1.10:22`. Because no network address translation (NAT) occurs on the bastion, App-B receives packets directly stamped with source IP `10.10.1.10`.

3. Why does Active peering not guarantee a successful ping?

An active peering connection is solely a Layer-3 conduit. Successful communication requires: 1. Valid route entries in both VPC route tables. 2. Stateful Security Group rules allowing inbound ICMP. 3. Stateless Network ACL rules permitting bidirectional traffic. 4. An active operating system kernel configured to reply to ICMP.

4. Can VPC B use VPC A's internet gateway over peering?

No. AWS VPC Peering is non-transitive and prohibits "edge-to-edge" routing. Traffic cannot enter a VPC via peering and exit through an Internet Gateway or NAT Gateway. To provide shared egress across multiple VPCs, an AWS Transit Gateway architecture is required.

---

8. Responsible Cleanup & Evidence

To adhere to responsible cloud stewardship and eliminate unnecessary AWS costs:
1. **EC2 Compute:** Terminated `ewb-bastion`, `ewb-app-a`, and `ewb-app-b`. Confirmed root EBS volumes were deleted.
2. **VPC Peering:** Deleted `ewb-a-b-peer`, which automatically removed the associated routing targets.
3. **Security Groups:** Deleted custom security groups `ewb-sg-bastion`, `ewb-sg-app-a`, and `ewb-sg-app-b`.
4. **Subnets & Route Tables:** Removed custom subnets and dissociated route tables.
5. **Gateways & VPCs:** Detached and deleted `ewb-igw-a`. Deleted `ewb-vpc-a` and `ewb-vpc-b`.

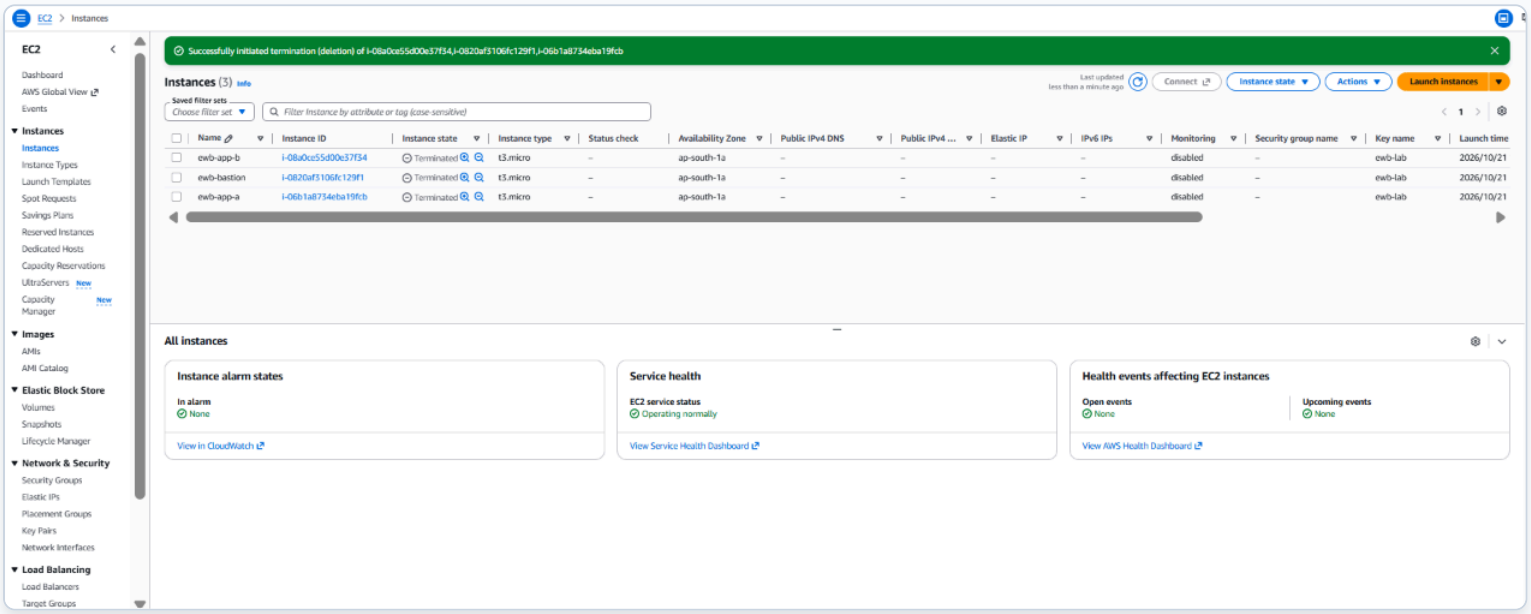


Figure 10: Teardown & Cost Elimination Verification

Host: AWS Console (EC2 & VPC) | **Test:** Resource Clean-up Audit | **Result:** Instances Terminated, Peering Deleted, Zero Chargeable Resources

---

9. Key Learnings & Takeaways

- Learned how to plan non-overlapping CIDR blocks to prevent routing collisions. - Mastered secure bastion jump host administration using OpenSSH `ProxyJump`. - Understood the difference between stateful Security Groups and stateless Network ACLs. - Gained hands-on experience designing and defending enterprise AWS VPC architectures.